



ECEN 260 - Laboratory Final Project

Greenhouse/Hydroponics Monitor

Shaymon Madrid

Instructor: Brother Brother Lynn Watson
4/2/2026
BYU-Idaho Semester: Winter 2026

Contents

1	Lab Overview	3
1.1	Objectives	3
2	Specifications	4
2.1	Project Requirements	4
2.2	Project Requirements	4
2.3	Operating Procedures/Setup	5
2.3.1	GPIO	5
2.3.2	ADC and Other	5
2.4	Known Limitations	6
2.5	Parts List	6
3	Design Essay	7
4	Schematics	8
5	Test Plan and Test Results	9
5.1	Demo Video link	12
6	Code	13
6.1	Code selection from main.c	13
7	Visuals	15
8	Conclusion	17

List of Figures

1	Complete Circuit	8
2	An image of the complete system	16

1 Lab Overview

This report is for the Final Project I made for the ECEN 260 course at Brigham Young University–Idaho. The aim of this lab is to take several of the topics we have learned previously, apply them to a single project that fits a specific need.

1.1 Objectives

- Monitor important environmental conditions, including temperature, humidity, and water level.
- Provide recent sensor readings to the user on an LCD screen.
- Allow the user to cycle through multiple measurement displays using a pushbutton input.
- Maintain periodic sensor updates so the displayed data remains current.
- Demonstrate integration of multiple embedded system concepts, including GPIO interrupts, ADC, digital sensor communication, timers, and display interfacing.

2 Specifications

This system is designed to monitor environmental conditions in a greenhouse or hydroponics setting. The project scope and course-driven requirements are listed in Section 2.2.

The system includes several input and output devices that allow both environmental monitoring and user interaction. It shall measure ambient temperature, humidity, and water level. Sensor data shall be updated periodically, regardless of which measurement is currently displayed on the LCD, so that each displayed reading reflects a recent measurement. The user shall be able to press a button to cycle through the temperature/humidity, and water level readings shown on the LCD. Programmed in to the project is also the capability to attach a fan. The project shall be able to use a GPIO Output to toggle a fan to turn on or off, such as through a transistor.

2.1 Project Requirements

The project must apply engineering design principles to integrate three or more of the following major course concepts:

- **GPIO Digital External Interrupts** (timer events are not external interrupts)
- **ADC** inputs (either polling or sampling on a timer)
- **PWM** outputs
- **Timers** (used for something other than ADC sampling or PWM outputs)
- **UART** communication between the micro-controller and a laptop or between two micro-controllers
- **Display** using SPI or I2C communication
- **Digital Communication** with another I/O device, such as a sensor or controller, using a protocol such as I2C, UART, or SPI

This project satisfies these requirements through the use of a pushbutton configured as a GPIO external interrupt, a water sensor read through the ADC, an LCD used as the system display, and a DHT11 temperature and humidity sensor used for digital communication. In addition, timing control is required for communication with the DHT11 sensor so that the micro-controller can correctly receive sensor data and an output signal can be sent through a GPIO pin to turn on or off a fan.

2.2 Project Requirements

The project should apply engineering design principles to integrate three or more of the following major concepts from the course:

- **GPIO Digital External Interrupts** (note: timer events are not external interrupts)
- **ADC** inputs (either polling or sampling on a Timer)

- **PWM** outputs
- **Timers** (used for something other than ADC sampling or PWM outputs)
- **UART** communication between your microcontroller and Laptop or between two microcontrollers
- **Display** (with SPI or I2C communication)
- **Digital Communication** with a different I/O device, such as a sensor or controller, using a protocol such as I2C, UART, or SPI (note: reading/writing to buttons/LEDs doesn't count for this)

This project makes use of a button for a GPIO Digital External Interrupt, a water sensor using ADC, an LCD Display, and a DHT11 Temperature and Humidity sensor for Digital Communication. This DHT11 sensor also makes use of a timer to handle necessary delay for the device to know when it can send data to the microprocessor. Additionally, an RGB LED was included to be a visual for the user on the temperature and the capability of adding a fan to the system.

2.3 Operating Procedures/Setup

- Configure microcontroller as I2C master device as follows:
 - PB8 as I2C1_SCL.
 - PB9 as I2C1_SDA.
 - Change mode to I2C under connectivity tab.
 - Enable PB8 and PB9 pull-up resistors.
- Assign and initialize the following (using CubeMX):

2.3.1 GPIO

- * PC13, TEMP_SENSOR_BUTTON, as GPIO_EXTI13 with mode as Interrupt with Falling edge trigger.
- * PA5, LD2[green Led], as GPIO_Output. Level set to Low, mode as Output Push Pull.
- * PA1, TEMP_SENSOR, as GPIO_Input with no pull resistors.

2.3.2 ADC and Other

- * PA0, WATER_SENSOR, as ADC1_IN5. In ADC1, set IN5 to single-ended mode
- * PC1, FAN, as GPIO_OUT.
- *

2.4 Known Limitations

- The sensors are sensitive to poor electrical connections which can cause the system to display 'garbage data'.
- Only two display states are currently possible.
- There is no calibration set-up for the water sensor, just a direct conversion to a percentage value.
- If the DHT11 read fails, the system only reports *Sensor Error!* on the LCD. It does not attempt retries, fault recovery, or any more detailed error reporting, so troubleshooting sensor failures would be limited.
- The DHT11 provides integer-only temperature and humidity values in this implementation. Decimal precision from higher-quality sensors is not supported, and the DHT11 itself is also a relatively low-accuracy sensor.
- There is no data logging, communication to another device, or persistent storage. The system only displays the most recent readings locally on the LCD.
- When using a fan with the system, the current pull can become too large to support the rest of the system.

2.5 Parts List

- Breadboard
- STM32 Nucleo-476RG Microcontroller
- Jumperwires and Resistors
- Laptop/Computer and USB cable
- 1602 Character LCD with I2C daughterboard
- DHT11 Humidity/Temperature Sensor
- RGB LED
- DC Fan

For specifics on the wiring setup, see the schematics in Section 4.

3 Design Essay

This project is a hydroponics/greenhouse environmental monitoring system designed to meet the need for accurate and timely monitoring of growing conditions. In hydroponic and greenhouse systems, plant health depends on maintaining proper conditions such as temperature, humidity, and water level. If these conditions are not monitored carefully, crops can be damaged, growth can become inconsistent, and resources can be wasted. This system collects data from sensors, processes that data, and displays it in a clear and understandable format so the user can make better decisions about maintaining the growing environment. In the future, the system could be expanded to include devices such as fans and water pumps, along with additional sensors such as pH and particulate sensors.

This design also considers public health, safety, welfare, and broader global, cultural, social, environmental, and economic factors. From a health and welfare perspective, improved monitoring supports healthier crop production by maintaining stable conditions and reducing the risk of poor-quality yields. From a safety perspective, monitoring water levels and environmental conditions can help prevent failures that could damage equipment or create hazards. Environmentally, the system promotes efficient use of water and other resources, which is one of the major benefits of hydroponics. Economically, it can reduce crop loss and improve productivity by helping users identify problems early. On a broader level, systems like this can support more sustainable agriculture and more reliable food production in different communities and settings.

Three major course concepts integrated into this design are analog-to-digital conversion (ADC), LCD display interfacing, and external interrupts. ADC allows analog sensor signals to be converted into digital data that the micro-controller can process. LCD interfacing makes it possible to present sensor readings in a clear and readable way for the user. External interrupts improve system responsiveness by allowing important events to be handled immediately instead of relying only on constant polling. In addition, the DHT11 sensor demonstrates digital communication by sending temperature and humidity data directly to the controller. There is also an RGB LED using PWM to be an additional indicator to the user of the temperature level of the system. Together, these concepts create a practical and effective environmental monitoring system.

4 Schematics

This section contains the schematic of the project. Note this was made in circuit maker where not all components are modeled which is why labels are what the wires connect to.

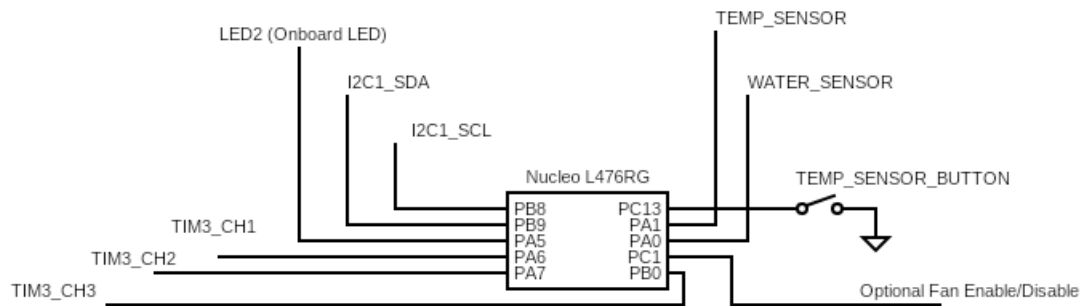


Figure 1: Complete Circuit

5 Test Plan and Test Results

Below are several test scenarios that we will make note of. We tested normal operation, boundary conditions, and common failure scenarios by observing whether the display output remained correct and stable. The scenarios are as follows below:

- **Test Scenario #1: Start-up/Splash Screen on Power-up**
 - **Step 1:** Power-on microprocessor.
 - **Step 2:** Wait for initial screen text and verify the display is correct.
 - Verify that the first line displays ECEN_260 and the second line displays FINAL_PROJ
 - **Expected result:** First line: ECEN_260, Second line: Final_Proj.
 - **Actual result:** The display initializes correctly, shows the project identification screen, and then moves into the live sensor display without freezing or showing corrupted characters.
- **Test Scenario #2: User Interaction (Push-Button)**
 - **Step 1:** Press onboard button once.
 - **Step 2:** Verify display changed to show data from the other sensor not previously being shown.
 - **Step 3:** Press onboard button again.
 - **Step 4:** Verify the display is showing data from the original sensor.
 - **Expected result:** Display switches between the two sensors and the data from them without issues.
 - **Actual Results:** Display switches between the two sensors and the data from them without issues
- **Test Scenario #3: Normal Sensor Monitoring**
 - **Step 1:** Verify that the water sensor continuously produced percentage-based readings.
 - **Step 2:** Press push-button to change sensor to be displayed.
 - **Step 3:** Verify that the displayed output is showing a temperature and humidity value.
 - **Expected result:** Water sensor data gives a percentage value on the LCD Display, DHT11 sensor data gives a temperature and humidity percentage on the LCD Display.
 - **Actual result:** Water sensor data gives a percentage value on the LCD Display, DHT11 sensor data gives a temperature and humidity percentage on the LCD Display.

- **Test Scenario #4: Water Sensor Response Under Changing Moisture Conditions**
 - **Step 1:** Place the water sensor in a dry condition and observe the displayed percentage.
 - **Step 2:** Expose the water sensor to a small amount of water and observe the display.
 - **Step 3:** Increase the amount of water contacting the sensor and observe the updated percentage value.
 - **Expected result:** Lower moisture levels produce lower displayed percentages, while increased water contact produces higher displayed percentages.
 - **Actual result:** The displayed water level percentage increased as more of the sensor was exposed to water, and the output remained stable and readable.
- **Test Scenario #5: Water Sensor Boundary Conditions**
 - **Step 1:** Test the water sensor while it is nearly dry.
 - **Step 2:** Observe the percentage shown on the LCD display.
 - **Step 3:** Test the water sensor while it is heavily exposed to water or submerged.
 - **Step 4:** Observe the percentage shown on the LCD display.
 - **Expected result:** The displayed water level remains within the valid range of 0% to 100% and does not produce invalid values.
 - **Actual result:** The displayed values stayed within the expected range and changed appropriately with sensor conditions.
- **Test Scenario #6: Temperature Response Test**
 - **Step 1:** Press the push-button until the temperature and humidity screen is displayed.
 - **Step 2:** Place a hand near the DHT11 sensor to slightly warm the surrounding area.
 - **Step 3:** Wait for the display to update.
 - **Step 4:** Observe the temperature reading on the LCD display.
 - **Expected result:** The displayed temperature should increase slightly and remain stable.
 - **Actual result:** The displayed temperature increased slightly after warming the sensor area and remained stable on the LCD display.
- **Test Scenario #7: Humidity Response Test**
 - **Step 1:** Press the push-button until the temperature and humidity screen is displayed.

- **Step 2:** Briefly breathe near the DHT11 sensor to increase the local humidity.
 - **Step 3:** Wait for the display to update.
 - **Step 4:** Observe the humidity reading on the LCD display.
 - **Expected result:** The displayed humidity should increase in response to the added moisture in the air.
 - **Actual result:** The displayed humidity increased after additional moisture was introduced near the sensor and remained readable and stable.
- **Test Scenario #8: Sensor Refresh While Displaying Another Sensor**
 - **Step 1:** Leave the LCD on the water sensor screen for several seconds.
 - **Step 2:** During that time, change the ambient temperature or humidity near the DHT11 sensor.
 - **Step 3:** Press the push-button to switch to the temperature and humidity screen.
 - **Step 4:** Observe whether the displayed values reflect the recent environmental change.
 - **Expected result:** Sensor values continue updating in the background, so the newly displayed screen shows recent measurements rather than stale data.
 - **Actual result:** The displayed values reflected recent sensor changes when switching screens, confirming that sensor updates continued even when that sensor was not currently being displayed.
- **Test Scenario #9: DHT11 Periodic Update Timing**
 - **Step 1:** Press the push-button until the temperature and humidity screen is displayed.
 - **Step 2:** Observe the sensor values continuously for several seconds.
 - **Step 3:** Note whether the values update periodically instead of changing every instant.
 - **Expected result:** The DHT11 sensor values update at a slower periodic interval and remain stable between updates.
 - **Actual result:** The temperature and humidity values updated periodically and remained stable between updates, showing correct timed sampling behavior.
- **Test Scenario #10: DHT11 Communication Failure**
 - **Step 1:** Disconnect the DHT11 sensor data connection or simulate a failed sensor response.
 - **Step 2:** Press the push-button until the temperature and humidity display screen is shown.
 - **Step 3:** Observe the message shown on the LCD display.

- **Expected result:** The LCD should display a sensor error message instead of invalid temperature or humidity values.
- **Actual result:**
The LCD displayed a sensor error message, confirming that the system handled the communication failure condition properly.
- **Test Scenario #11: LCD Display Stability**
 - **Step 1:** Run the system continuously while switching between the available display screens.
 - **Step 2:** Change the water level and the surrounding temperature or humidity during operation.
 - **Step 3:** Observe the LCD for flickering, leftover characters, or corrupted text.
 - **Expected result:** The LCD output remains stable, readable, and free of visual corruption while updating sensor data.
 - **Actual result:** The LCD output remained stable and readable during operation, and no corrupted or leftover characters were observed while the device was in a stable position. Moving wires caused corrupted and leftover characters to be observed.
- **Test Scenario #12: Extended Normal Operation**
 - **Step 1:** Allow the system to run continuously for an extended period of time.
 - **Step 2:** Periodically press the push-button to switch between display screens.
 - **Step 3:** Observe whether the displayed values remain correct and whether the system continues operating without freezing.
 - **Expected result:** The system continues monitoring and displaying sensor values correctly over time without unexpected resets or display errors.
 - **Actual result:** The system continued operating normally during extended testing, and the displayed values remained stable and correct.

5.1 Demo Video link

<https://youtu.be/BuJIDHrXCMw>

6 Code

The code below is what I consider to be the most critical portion of the program for this project. The reason for this is that these lines of code are the actual functions that allow the program to work. Functions such as DHT11_Start and DHT11_Update where we can actually use the sensors. This is especially important for the DHT11 sensor as it has some unique properties that must be accounted for due to it being a digital sensor that we communicate with. You may find the complete code in my GitHub repository here: [GitHub Repository](#)

6.1 Code selection from main.c

```
1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     if (GPIO_Pin == TEMP_SENSOR_BUTTON_Pin) // Match the label in CubeMX
4     {
5         buttonPressed = 1; // for button/display state machine code
6         HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin); // Uses the standard LD2
7         label
8     }
9 }
10 void delay_us(uint16_t us) {
11     __HAL_TIM_SET_COUNTER(&htim2, 0);
12     while (__HAL_TIM_GET_COUNTER(&htim2) < us);
13 }
14
15 uint8_t DHT11_Start(void) {
16     uint8_t Response = 0;
17     GPIO_InitTypeDef GPIO_InitStruct = {0};
18     GPIO_InitStruct.Pin = TEMP_SENSOR_Pin;
19     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
20     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
21     HAL_GPIO_Init(TEMP_SENSOR_GPIO_Port, &GPIO_InitStruct);
22     HAL_GPIO_WritePin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin, 0);
23     HAL_Delay(18);
24     HAL_GPIO_WritePin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin, 1);
25     delay_us(20);
26     GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
27     HAL_GPIO_Init(TEMP_SENSOR_GPIO_Port, &GPIO_InitStruct);
28     delay_us(40);
29     if (!(HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin))) {
30         delay_us(80);
31         if ((HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin)))
32             Response = 1;
33     }
34     while ((HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin)));
35     return Response;
36 }
37 uint8_t DHT11_Read(void) {
```

```

38     uint8_t i = 0, j;
39     for (j = 0; j < 8; j++) {
40         while (!(HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin)));
41         delay_us(40);
42         if (!(HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin)) i &=
~(1 << (7 - j)));
43         else i |= (1 << (7 - j));
44         while ((HAL_GPIO_ReadPin(TEMP_SENSOR_GPIO_Port, TEMP_SENSOR_Pin)));
45     }
46     return i;
47 }
48
49 void DHT11_Update(void)
50 {
51     if (DHT11_Start())
52     {
53         uint8_t rh_byte1 = DHT11_Read();
54         uint8_t rh_byte2 = DHT11_Read();
55         uint8_t temp_byte1 = DHT11_Read();
56         uint8_t temp_byte2 = DHT11_Read();
57         uint8_t checksum = DHT11_Read();
58
59         if (checksum == (uint8_t)(rh_byte1 + rh_byte2 + temp_byte1 +
temp_byte2))
60         {
61             Rh = rh_byte1;
62             Temp = temp_byte1;
63             dhtValid = 1;
64         }
65         else
66         {
67             dhtValid = 0;
68         }
69     }
70     else
71     {
72         dhtValid = 0;
73     }
74 }

```

7 Visuals

This section includes an image of the final system, not pictured here is an optional fan as the system would fail due to the current draw when it is operating. This is a link to an explanation video.

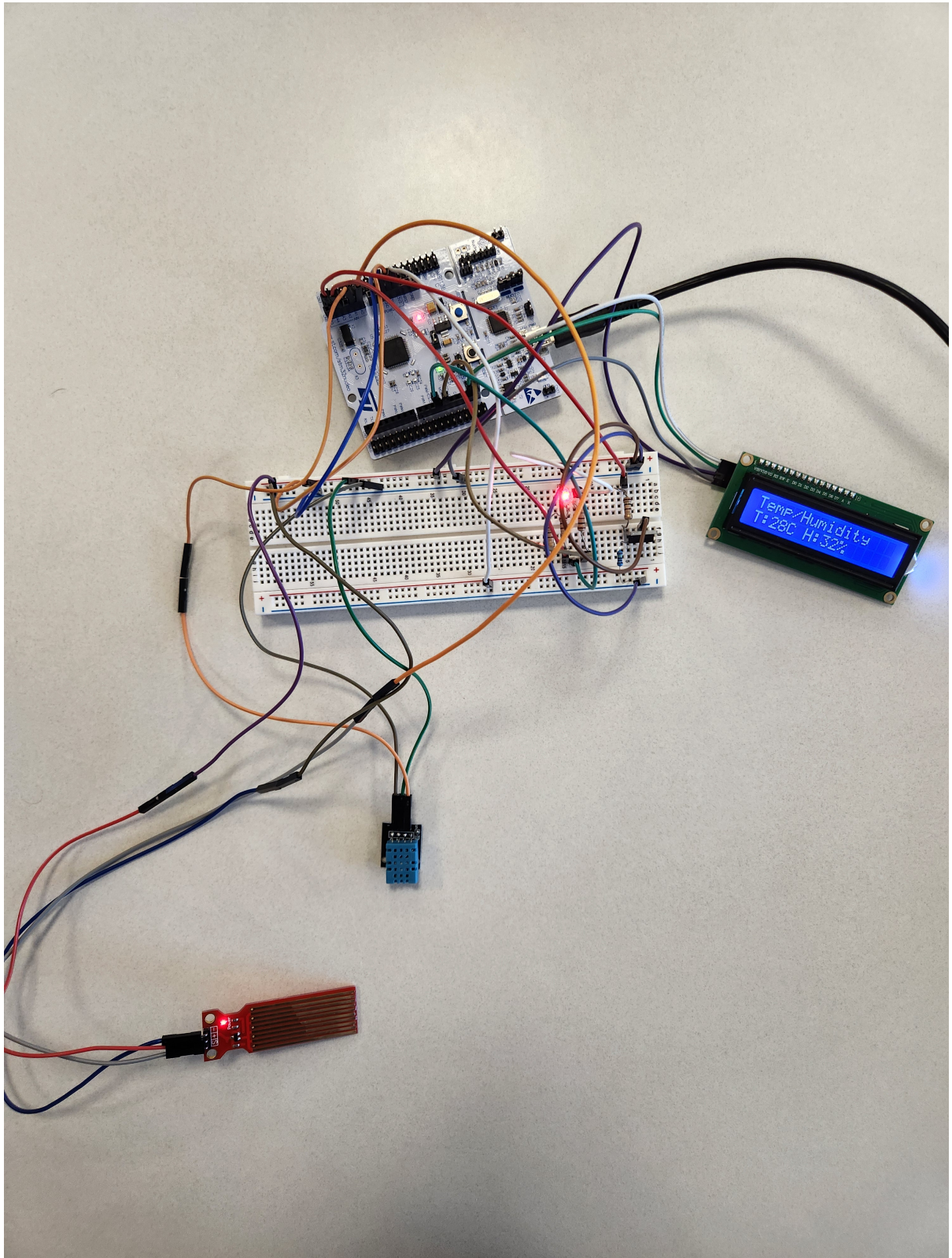


Figure 2: An image of the complete system

8 Conclusion

In this project, we successfully designed and implemented an embedded greenhouse and hydroponics monitoring system using the STM32 microcontroller. The system measured temperature, humidity, and water level while displaying the selected data on an I²C LCD. A pushbutton configured as an external interrupt allowed the user to cycle between display screens, while sensor values continued updating in the background so that the information shown remained current. The project also integrated digital communication with the DHT11 temperature and humidity sensor, ADC-based measurement for the water sensor, and GPIO control for additional output devices such as a fan and status LED. Throughout development, careful attention was required for timing, sensor communication, display formatting, and stable hardware connections to ensure reliable operation.

This project builds on earlier laboratory work by combining several major embedded systems concepts into a practical environmental monitoring application. Unlike previous labs that focused on individual peripherals or isolated communication protocols, this project required the integration of multiple subsystems including GPIO external interrupts, ADC input, digital sensor communication, timing control, and LCD output. It reinforced the importance of coordinating hardware and software so that sensor measurements, user input, and display updates could operate together in a responsive and organized way. Testing also showed that the system performed well under normal operation, changing environmental conditions, and several error scenarios, demonstrating both functionality and reasonable robustness for a prototype design. Overall, this project illustrates how embedded systems can be used to create a user-oriented monitoring platform for real-world applications such as greenhouse or hydroponic management.

References